Longest Common Subsequence

AlgoExpert · Hard · Dynamic Programming

The problem

- One LCS of two strings as a list of chars.

Intuition

- Match characters along two prefixes: a match extends the diagonal, otherwise take the better of dropping one character.

Brute force vs optimal

- Brute force: all subsequences of one string → $O(2^n \cdot m)$
- DP over prefixes → $O(n \cdot m)$
- Backtrack to recover the sequence

Approach

- 1. Build the LCS length table
- 2. Equal characters add one to the diagonal
- 3. Otherwise take the max of top or left
- 4. Backtrack to recover the letters

Worked example

			X	K	Y	K	Z	P	W
	0	0	0	0	0	0	0	0	
Z	0	0	0	0	0	1	1	1	
X	0	1	1	1	1	1	1	1	
V	0	1	1	1	1	1	1	1	
V	0	1	1	1	1	1	1	1	
Y	0	1	1	2	2	2	2	2	
Z	0	1	1	2	2	3	3	3	
W	0	1	1	2	2	3	3	4	

Fill: match → diagonal + 1, else max(up, left)

Worked example

			X	K	Y	K	Z	P	W
	0	0	0	0	0	0	0	0	
Z	0	0	0	0	0	1	1	1	
X	0	1	1	1	1	1	1	1	
V	0	1	1	1	1	1	1	1	
V	0	1	1	1	1	1	1	1	
Y	0	1	1	2	2	2	2	2	
Z	0	1	1	2	2	3	3	3	
W	0	1	1	2	2	3	3	4	

Backtrack from the bottom-right

Worked example

```
LCS = XYZW (length 4)
```

LCS = XYZW

Java solution

```java
public class LongestCommonSubsequence {
    public static List<Character> longestCommonSubsequence(String a, String b) {
        int n = a.length(), m = b.length();
        int[][] dp = new int[n + 1][m + 1];
        for (int i = 1; i <= n; i++) for (int j = 1; j <= m; j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1) ? dp[i - 1][j - 1] + 1 : Math.max(dp[i - 1][j], dp[i][j - 1]);
        List<Character> out = new ArrayList<>();
        int i = n, j = m;
        while (i > 0 && j > 0) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) { out.add(0, a.charAt(i - 1)); i--; j--; }
            else if (dp[i - 1][j] >= dp[i][j - 1]) i--; else j--;
        }
        return out;
    }
}
```

Complexity

- Time: O(n*m)
- Space: O(n*m)

Edge cases & pitfalls

- Off-by-one with the padding row/column
- Ties in backtracking: any choice valid
- Empty string → []

Interview tips & follow-ups

- Relate to edit distance and diff tools

Recap

- Match characters along two prefixes: a match extends the diagonal, otherwise take the better of dropping one character.
- Complexity: O(n*m)